

Multi-dimensional ($D \geq 4$) histograms in ROOT

September 18, 2025

In this ROOT exercise we cover the following class:

1. THnSparse (<https://root.cern.ch/doc/master/classTHnSparse.html>)

Regular histogram classes in ROOT, namely TH1, TH2, TH3, can be readily used for 1-dimensional, 2-dimensional, and 3-dimensional problems, respectively. However, already for 3-dimensional case the memory consumption becomes a serious concern. For instance, if one wants to book 3-dimensional histogram with double precision (8 bytes), using 10000 bins in each dimension, that might blow up all available memory on a computer, because $(10^5)^3 \times 8 \text{ B} = 8 \times 10^{15} \text{ B} = 8 \text{ PB}$ is a huge number. It is therefore essential to come up with another histogramming technology, which will allocate memory for bins only for those bins which are needed and filled. On the other hand, for empty bins memory shall not be allocated at all.

This can be achieved in ROOT with the class `THnSparse`. In this exercise, we demonstrate how to use these histograms in practice.

Declaration, filling and retrieving the content: *Use this code snippet and save it in the file named `sparse.C` — here it is demonstrated for 4-dimensional case, how `THnSparse` can be declared and used in a simple example. The whole procedure can be trivially generalized for more than 4 dimensions.*

```
// Include all headers:
#include <THnSparse.h>
#include <Riostream.h>

// Main:
int sparse() {

    // Declaration:
    const int nDim = 4;
    int nBins[nDim] = {10, 20, 30, 40}; // number of bins in each dimension
    double min[nDim] = {0., 100., 1000., 2000.}; // lower bin boundary in each dimension
    double max[nDim] = {10., 120., 1030., 2040.}; // upper bin boundary in each dimension
    THnSparse* hs = new THnSparseD("hs", "someTitle", nDim, nBins, min, max);

    // Filling:
    double vector[nDim] = {5., 115.3, 1022.4, 2032.7}; // values to be filled
    hs->Fill(vector); // fill the above vector with default weight 1
    hs->Fill(vector, 2.); // fill the above vector with weight 2

    // Retrieve the content from particular bin:
    int binNumber[nDim] = {5, 15, 22, 32}; // specify bin number in each dimension
    cout << hs->GetBinContent(binNumber) << endl;

    return 0;
}
```

Execute the above code either in the interpreted mode via `root -q sparse.C`, or compile and execute in the standard way using `root -q sparse.C++`. In both cases, you shall get 3 as a printout, because in the specified multi-dimensional bin, the multi-dimensional vector was filled 3 times (once with default weight 1, and once with weight 2). All other bins shall be empty.

Projections to lower dimensions: *Use this code snippet and save it in the file named `projection.C` — here it is demonstrated how one 3-dimensional sparse histogram can be projected onto three 2-dimensional sparse histograms. The procedure can be trivially generalized for other cases of interest:*

```
// Include all headers:
#include <THnSparse.h>
#include <Riostream.h>

// Main:
int projection() {

    // Declaration of 3-dimensional sparse histogram:
    const int nDim = 3;
    int nBins[nDim] = {10, 20, 30}; // number of bins in each dimension
    double min[nDim] = {0., 100., 1000.}; // lower bin boundary in each dimension
    double max[nDim] = {10., 120., 1030.}; // upper bin boundary in each dimension
    THnSparse* hs_xyz = new THnSparseD("hs_xyz", "someTitle", nDim, nBins, min, max);

    // Projections onto three 2-dimensional sparse histograms:
    // a) project out "2 = z", i.e. keep "0 = x" horizontal vs. "1 = y" vertical
    int dimensions_xy[2] = {0,1};
    THnSparse *proj_xy = hs_xyz->Projection(2, dimensions_xy);

    // b) project out "1 = y", i.e. keep "0 = x" horizontal vs. "2 = z" vertical
    int dimensions_xz[2] = {0,2};
    THnSparse *proj_xz = hs_xyz->Projection(2, dimensions_xz);

    // c) project out "0 = x", i.e. keep "1 = y" horizontal vs. "2 = z" vertical
    int dimensions_yz[2] = {1,2};
    THnSparse *proj_yz = hs_xyz->Projection(2, dimensions_yz);

    return 0;

}
```

Challenge: *Set up a Toy Monte Carlo study, in which you will fill 3-dimensional sparse histogram (decide yourself on the binning and total number of samplings). After the 3-dimensional sparse histogram is filled, make projection onto one 2-dimensional sparse histogram by keeping axes “x” and “y”, and onto one 1-dimensional sparse histogram by keeping only the axis “y”. This example illustrates how one can during data processing book and fill one large multi-dimensional sparse histogram, and then later “cherry-pick” from it only the information which is needed in particular analysis, by making projections to lower dimensions.*